

Fee Schedule PDF Ingestion Platform

Project Spec | Single-Tenant Internal Tool | v1.0

1. Problem Statement

Dental groups receive fee schedules from insurance carriers (Metlife, Cigna, Aetna, Delta Dental, United Concordia, and others) as large, multi-page PDFs. Each PDF lists hundreds to tens of thousands of procedure codes (CDT codes), procedure descriptions, allowed amounts, and contract-specific modifiers.

Today this data is keyed into our practice-management systems by hand, or pasted from Excel files when a carrier happens to provide one. That manual process is slow (often days per carrier per region), error-prone, and blocks billing accuracy and revenue-cycle analytics.

We need an internal tool that ingests a fee-schedule PDF and writes a clean, queryable representation of every row into Postgres, with visibility into how the parse went.

2. Goals and Non-Goals

Goals

- Accept large fee-schedule PDFs (hundreds of pages, tens of MB) via a web upload.
- Parse the PDF into structured rows using Google's Gemini LLM as a generic, layout-agnostic extractor.
- Persist results to Postgres in a normalized schema suitable for downstream querying and joins.
- Surface real-time parse status, throughput, row counts, and elapsed time in the UI.
- Run the entire stack as a containerized service on Azure Container Apps.

Non-Goals (v1)

- Multi-tenant isolation or external customer access. This is an internal single-tenant tool.
- Authentication and role-based access control beyond a basic shared SSO gate. Deferred.
- Carrier-specific template tuning. The generic Gemini-driven parser is the only path in v1.
- Two-way sync with Denticon, Cloud 9, or Dental OS. Downstream consumers will read from Postgres.
- OCR-quality remediation of poorly scanned PDFs. We assume vector or clean-scanned PDFs.

3. Users and Scenarios

The primary user is an internal operations or revenue-cycle analyst.

Primary scenarios

- An analyst receives a new Metlife PPO fee schedule for California, drags the PDF into the uploader, and watches a progress page until ingestion is complete. They click through to view the resulting rows and export the table.

- A re-ingest scenario where a corrected PDF from the carrier supersedes an earlier load. The user uploads again and is shown a diff or supersede option.
- An engineer audits why a particular row failed to extract, looking at the per-page parse log and the Gemini response trail.

4. Success Metrics

- **Time-to-ingest:** a 500-page fee-schedule PDF completes end-to-end in under 10 minutes at p95.
- **Row accuracy:** at least 98% row-level field accuracy on a labeled sample of 5 carriers, measured against a hand-verified gold set.
- **Operational visibility:** every parse run records page count, row count, elapsed time, token usage, and a per-page status (parsed, partial, failed).
- **Adoption:** replaces at least 80% of manual fee-schedule entry within one quarter of launch.

5. High-Level Architecture

The system is a single-tenant web application deployed to Azure Container Apps. There are three logical services, each as its own container:

- **Web frontend.** React + TypeScript single-page app, served by an Nginx sidecar or directly from a small Node server. Talks to the API over HTTPS.
- **API service.** TypeScript (Node + Fastify or NestJS). Owns upload intake, job orchestration, parse-status APIs, and query APIs over the parsed data. Built using Antigravity as the AI-assisted IDE for the engineering team.
- **Parse worker.** TypeScript worker that pulls jobs from a queue, streams PDF pages to Gemini, normalizes the structured response, and writes rows to Postgres in batches. Scales horizontally on Azure Container Apps using KEDA-driven scale rules.

Supporting infrastructure

- **Azure Blob Storage** for the raw uploaded PDFs and intermediate page images or chunks.
- **Azure Postgres Flexible Server** for the canonical fee-schedule data and job metadata.
- **Azure Service Bus** (or Postgres-backed queue in v1) for the parse job queue.
- **Azure Container Registry** for container images.
- **Azure Front Door / Application Gateway** for TLS termination and routing.
- **Gemini API** (via Google AI Studio or Vertex AI) for document parsing.

Request flow

- User uploads a PDF in the React UI. The browser requests a SAS upload URL from the API and uploads directly to Blob Storage.
- Once upload completes, the browser calls POST /jobs with the blob reference. The API creates a parse_job row and enqueues a message.
- A parse worker picks up the message, downloads the PDF, splits it into page-sized chunks, sends each chunk to Gemini with a strict JSON schema prompt, validates the response, and inserts rows into fee_schedule_rows in batches of 500 inside transactions.
- The worker emits progress events (page completed, rows inserted, errors) by updating parse_job_progress. The API exposes these via SSE so the UI can render a live progress bar.
- On completion, the job is marked done and the UI shows summary stats and links to the parsed table.

6. Data Model (Postgres)

Three core tables, kept deliberately simple in v1:

parse_job			
id	uuid	pk	
source_filename	text		
blob_path	text		
carrier_hint	text		nullable, user-provided
status	text		queued running completed failed
page_count	int		populated on parse start

row_count	int	updated incrementally
bytes	bigint	
started_at	timestampz	
completed_at	timestampz	
duration_ms	int	materialized on completion
gemini_input_tokens	bigint	
gemini_output_tokens	bigint	
error_summary	text	nullable

parse_job_progress

job_id	uuid fk -> parse_job.id	
page_number	int	
status	text	parsed partial failed
rows_extracted	int	
error	text	nullable
parsed_at	timestampz	
primary key (job_id, page_number)		

fee_schedule_rows

id	bigserial pk	
job_id	uuid fk -> parse_job.id	
carrier	text	inferred or user-provided
plan_name	text	nullable
effective_date	date	nullable
region	text	nullable
cdt_code	text	indexed
procedure_desc	text	
allowed_amount	numeric(12,2)	
modifier	text	nullable
source_page	int	
raw_row	jsonb	original Gemini output for audit

Indexes: (carrier, cdt_code), (job_id), GIN on raw_row for ad-hoc audit queries. Job history and per-page status are retained indefinitely. Raw PDFs in blob storage follow a 90-day lifecycle to control cost.

7. Parsing Pipeline

The parser is generic and Gemini-driven, with no carrier-specific templates in v1. For each PDF the worker:

- Downloads the file from Blob Storage to a local tmpfs.
- Uses `pdfjs-dist` or `pdf-lib` to determine page count and split into per-page renderings. For text-bearing PDFs we pass the extracted text plus the page image to Gemini; for image-only pages we send the image alone.
- Calls Gemini (model: `gemini-2.5-pro` for accuracy, with `gemini-2.5-flash` as a configurable fallback for cost) using a structured-output prompt that pins the response to a JSON schema matching `fee_schedule_rows` columns. Temperature is set to 0.
- Validates the response with Zod. Rows failing validation are bucketed into the `partial` state for that page rather than crashing the job.
- Buffers validated rows and flushes to Postgres every 500 rows or every 5 seconds, whichever comes first, using a single parameterized multi-row INSERT.
- Emits a `page_completed` event with row count and elapsed time per page.

****Concurrency.**** A single worker processes up to 4 pages in parallel against Gemini, subject to per-minute token quotas. Multiple workers can run side-by-side; jobs are sticky per worker but pages within a job parallelize.

****Retries.**** Per-page Gemini calls retry up to 3 times with exponential backoff on transient errors. Persistent failures mark the page `failed` and continue. The job completes as long as at least one page parsed; the UI clearly indicates partial parses.

8. User Experience

The UI is intentionally narrow.

Upload page

Drag-and-drop zone, optional carrier hint dropdown (Metlife, Cigna, Aetna, Delta Dental, United Concordia, Other), optional plan name and effective date fields, and a Start button. While uploading we show a determinate progress bar fed by Blob Storage upload events.

Job detail page

Live status board for one parse job. Shows:

- Filename, size, page count, carrier.
- Status pill (Queued, Running, Completed, Failed).
- Elapsed time, ETA (computed from average page parse time so far).
- Rows ingested so far, updating live via SSE.
- A per-page strip showing parsed / partial / failed colors so the user can spot trouble pages.
- On completion, a summary card with total rows, total duration, total tokens used, and a CTA to view the rows table.

Rows browser

A paginated, filterable table over `fee_schedule_rows` for the job. Columns: CDT code, description, allowed amount, modifier, source page. CSV export.

Jobs list

Historical view of all parse jobs with status, row count, and duration. Click through to detail.

9. Non-Functional Requirements

- **Performance.** 500-page PDF completes in under 10 minutes at p95 with one worker; under 5 minutes with two workers.
- **Throughput.** Each worker container sustains 50 to 100 rows per second of insert throughput against Azure Postgres.
- **Reliability.** A worker crash mid-job does not lose state. The job resumes from the last committed page based on `parse_job_progress`.
- **Observability.** Structured logs to Azure Log Analytics. App Insights for traces. Dashboard with job throughput, error rate, and Gemini token spend.
- **Security.** All ingress over HTTPS. SAS tokens for blob upload scoped to a single blob and 15-minute TTL. Postgres credentials in Azure Key Vault, mounted as env vars. Network egress to Gemini API only.
- **Cost guardrails.** Per-job token cap (configurable, default 5M output tokens). A hard stop with a clear error message if a job would exceed the cap.

10. Hosting and Deployment

All services run on Azure Container Apps in a single environment.

- **Frontend container:** 0.25 vCPU / 0.5 GiB, min 1 replica.
- **API container:** 0.5 vCPU / 1 GiB, min 1, max 5, HTTP-scaled.
- **Worker container:** 1 vCPU / 2 GiB, min 0, max 10, KEDA-scaled on queue length.

****CI/CD**** via GitHub Actions: on merge to main, build images, push to ACR, and deploy via `az containerapp update`. Database migrations run as a pre-deploy job using a migration container (e.g., Prisma Migrate or node-pg-migrate).

****Environments:**** dev and prod, each with its own resource group and Postgres instance. Container Apps revisions provide rollback.

11. Engineering Approach

The engineering team will build this using Antigravity as the AI-assisted IDE, leaning on its agentic workflows to scaffold the React + TypeScript app, generate the Postgres schema migrations, and produce the worker pipeline. Gemini is the runtime LLM dependency for the parsing pipeline; Antigravity is a development-time accelerator.

12. Milestones

A six-week plan to v1.

Week	Focus
1	Scaffolding. React app, API skeleton, Postgres schema and migrations, Container Apps infra via Bicep. Hello-world parse of a 5-page sample.
2	Upload flow end-to-end. Blob SAS, job creation, queue, worker stub. Live SSE progress.

3	Gemini integration with structured output, Zod validation, batched inserts. First real Metlife PDF parses.
4	UI polish: jobs list, rows browser, CSV export, per-page status strip. Token accounting.
5	Hardening. Retries, partial-failure handling, observability, cost caps, security review.
6	Gold-set evaluation across 5 carriers, accuracy tuning of prompts, launch to internal users.

13. Risks and Open Questions

- **Gemini accuracy on dense tables.** Carrier PDFs vary wildly. We mitigate with structured output and Zod validation, but accuracy below 98% on a carrier would force a per-carrier template fallback that is currently out of scope. We will gate launch on the gold-set result.
- **Token cost.** A large multi-thousand-page PDF could be expensive. The per-job cap and a Flash-tier fallback are the levers; we will track cost per row as an operating metric.
- **PDF quality.** Scanned PDFs without an OCR layer will degrade output. We treat OCR remediation as a v2 concern but should detect and warn at upload time.
- **Schema drift in Postgres.** As real carrier PDFs surface fields we did not anticipate (bundled-code rules, tiered allowed amounts), we may need to extend the schema. The `raw_row` JSONB column buys flexibility without a migration in the short term.
- **Single-tenant assumption.** If product wants to expose this to external dental groups later, we will need to retrofit tenant isolation, row-level security, and per-tenant rate limits. Worth noting now to keep code paths additive.

14. Out-of-Scope Follow-Ons (v2+)

Per-carrier template tuning, multi-tenant + auth, OCR for scanned PDFs, diff/supersede across re-ingests, direct push into Denticon and Cloud 9, and analyst-driven correction UI for partial rows.